# AZURE MEETUP PUGLIA

BY COMMUNITY • FOR COMMUNITY

#GLOBALAZURE

2026

# BENVENUTI

# GLOBAL AZURE PUGLIA 2026

Venerdì 17 aprile - Bari

# Una CLI per domarli tutti: il potere di azd!!

**Massimo Bonanni**

***Senior ~~Cloud~~ Technical Trainer***

Microsoft CLOWN

Global Azure Puglia 2026 – 17 aprile

# Grazie Agli Sponsor

AZURE MEETUP PUGLIA

## Sponsor Gold

UGOLOPEZ.IT

## Sponsor Silver

CODE architects — Tomorrow's software, today

eusoft — more than a ums

AZURE MEETUP PUGLIA

#GLOBALAZURE 2026 · BY COMMUNITY · FOR COMMUNITY

Global Azure Puglia 2026 – 17 aprile

global AI community

# What is Azure Developer CLI?

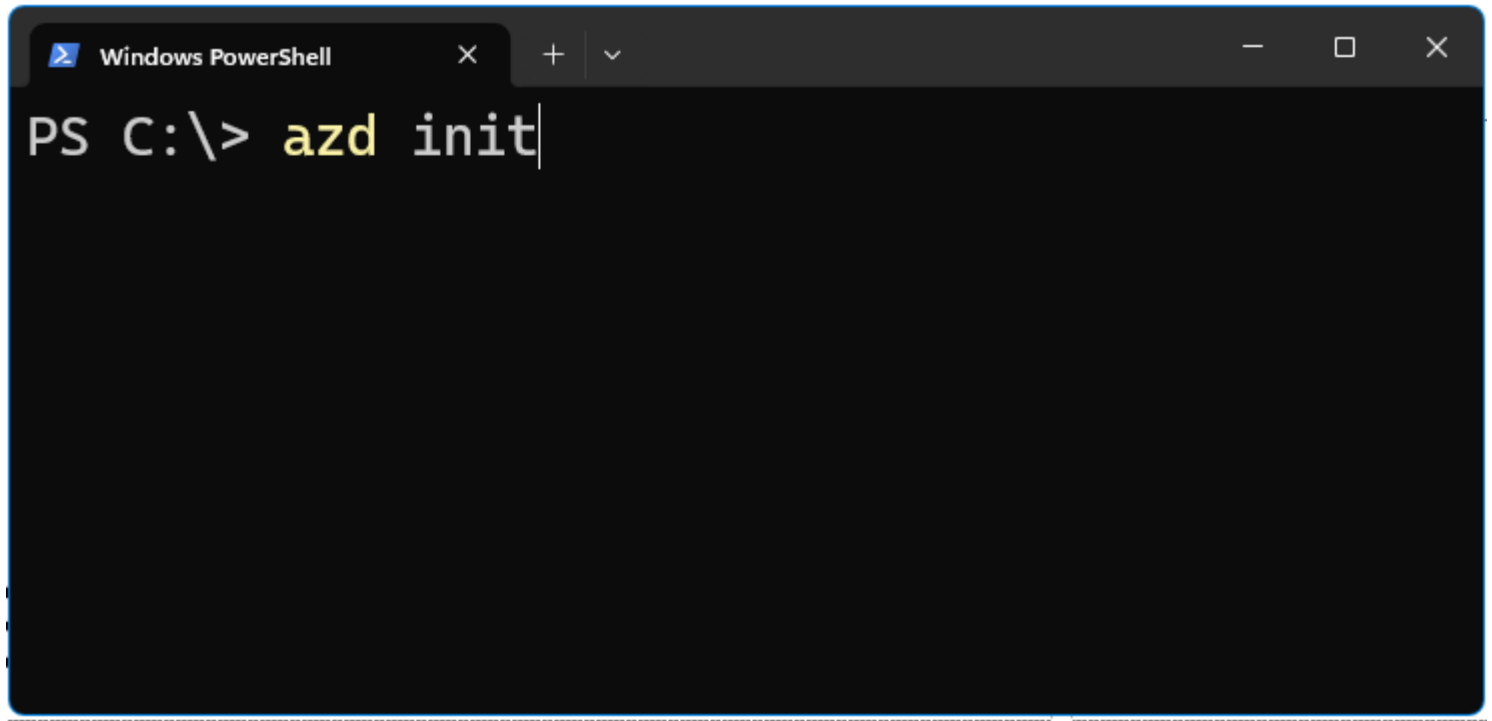
An open-source tool that accelerates provisioning and deploying app resources on Azure

## Accelerate Deployment

Provision and deploy app resources on Azure with a single open-source CLI tool. Go from code to cloud in minutes, not hours.

## Developer-Friendly Workflow

Best-practice commands map to every stage of your dev workflow — terminal, IDE, or CI/CD pipelines. One tool, every environment.

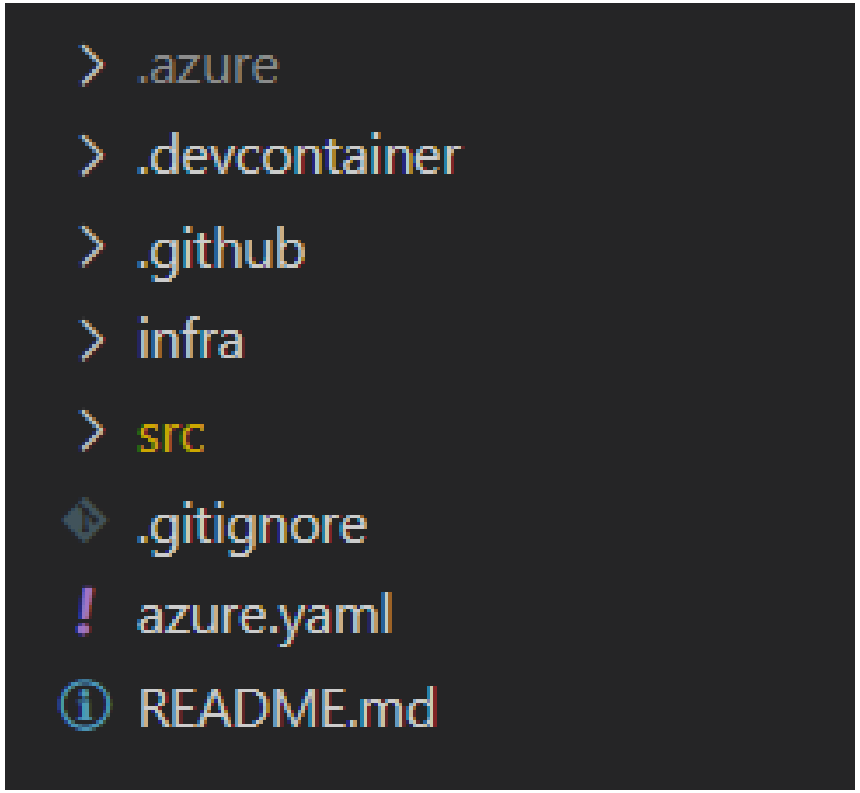

Windows PowerShell
PS C:\> azd init

# AZD initialization

The `azd init` command supports several workflows to prepare your app to work with azd:

| Scan current directory | Select a template | Create a minimal project |
| --- | --- | --- |
| Analyzes an existing app codebase to generate appropriate azd configuration files and resources | Clones and initializes a template from a GitHub repo.<br>Awesome AZD repo provides you several templates to start with. | Initializes a basic `azure.yaml` file as a starting point for building your own azd template from scratch. |

# AZD template

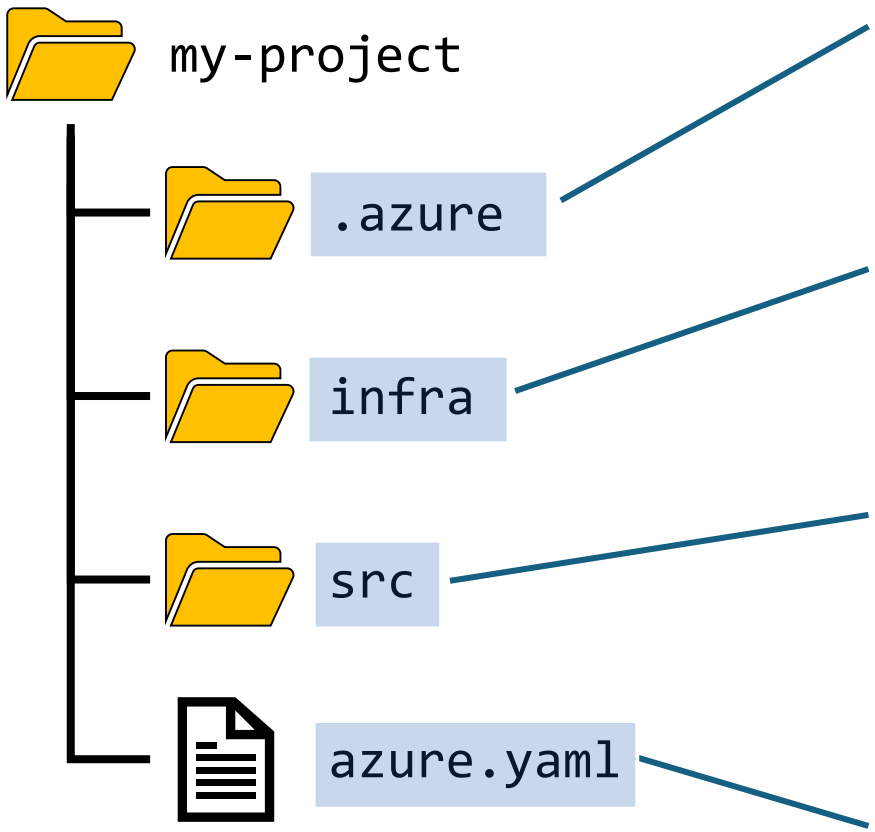



AZD template is a regular code repository that include:

- Application code
- Infrastructure files
- Configuration files

AZD templates enable you to provision Azure resources, deploy your application, configure CI/CD pipelines, and more.

# Template structure



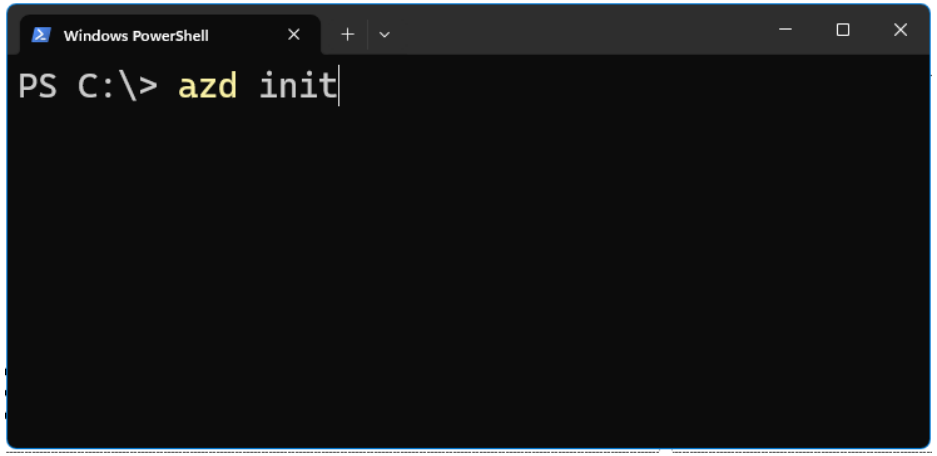

Contains environments configurations, such as the location to deploy resources.

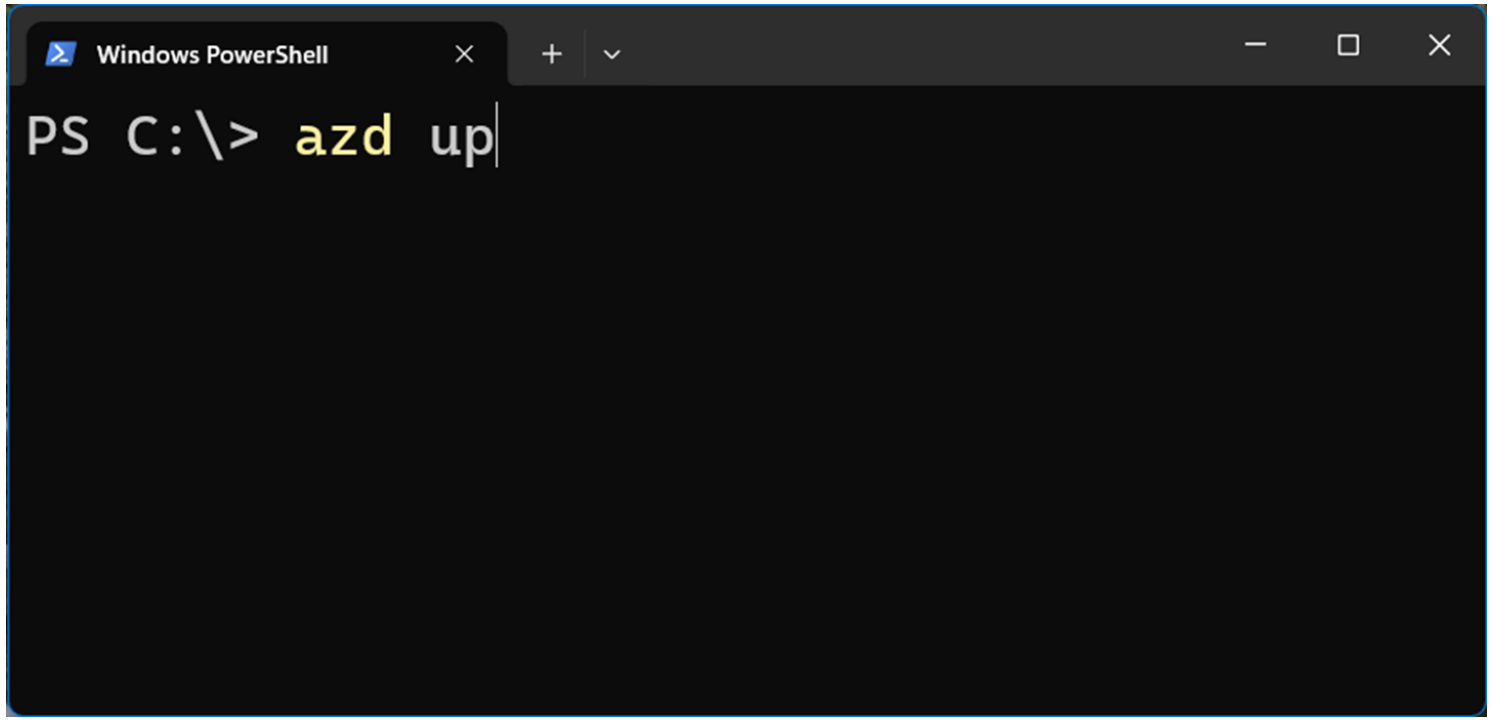
Includes all Bicep or Terraform infrastructure-as-code files for the azd template, which azd uses to set up the necessary Azure resources for your app.

Contains the app source code. azd packages and deploys the code based on configurations in azure.yaml.

A configuration file links your project's source code folders to Azure resources in the infra folder, enabling deployment of services like an API and web front-end to separate Azure resources.

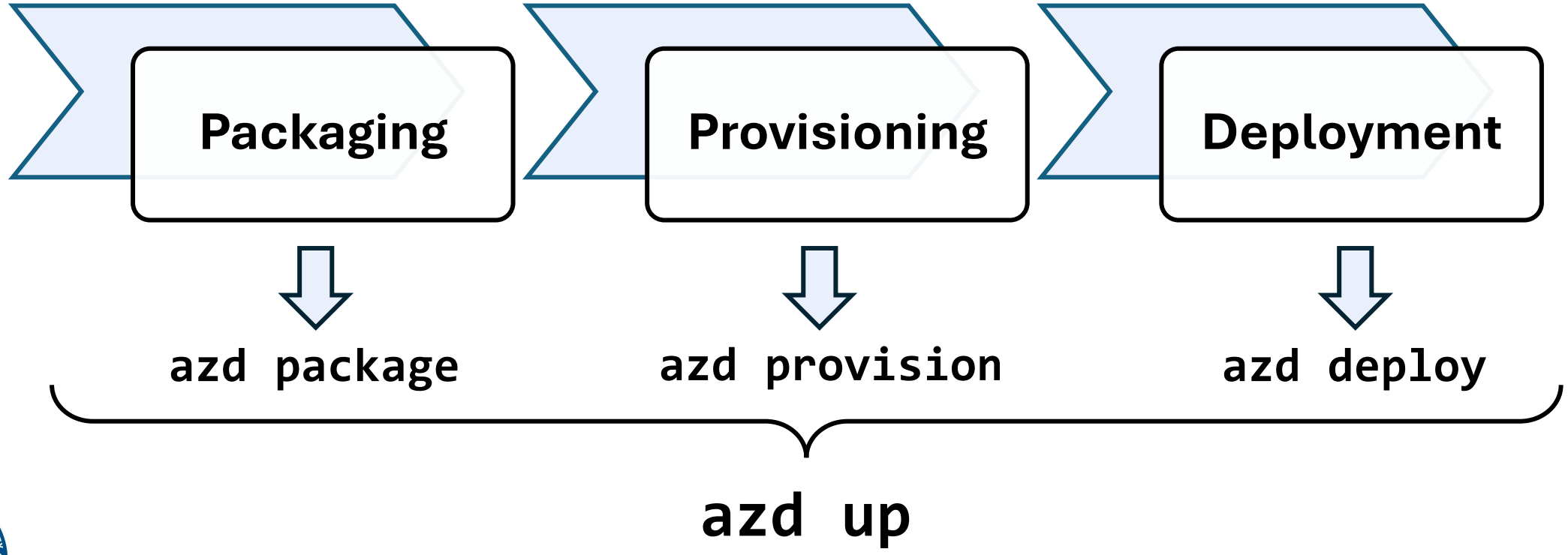
# Demo




Windows PowerShell
PS C:\> azd up

# azd up workflow

AZD enables you to **provision** and **deploy** application resources on Azure with only a single command using **`azd up`**.



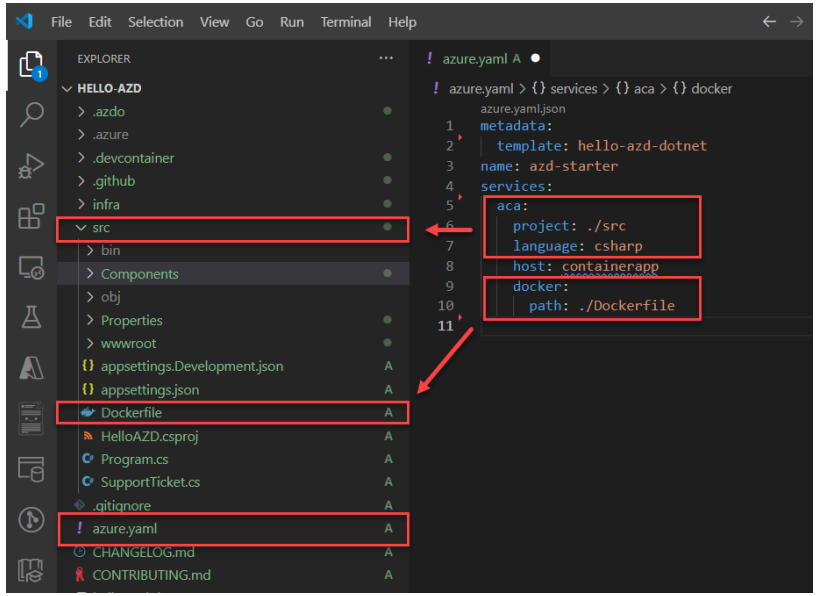
# Packaging stage

Packaging can involve building or compiling code, bundling dependencies, or creating deployment artifacts such as Docker images.

The azd template structure typically includes a `src` folder where the application code resides. Build scripts or configuration files (such as a Dockerfile) can influence how the application should be packaged.

The `azure.yaml` file contains configuration mappings that tell azd where your app code lives and which language it uses so azd can package it appropriately.

This stage ensures that the application is in a deployable state before moving to the next step.



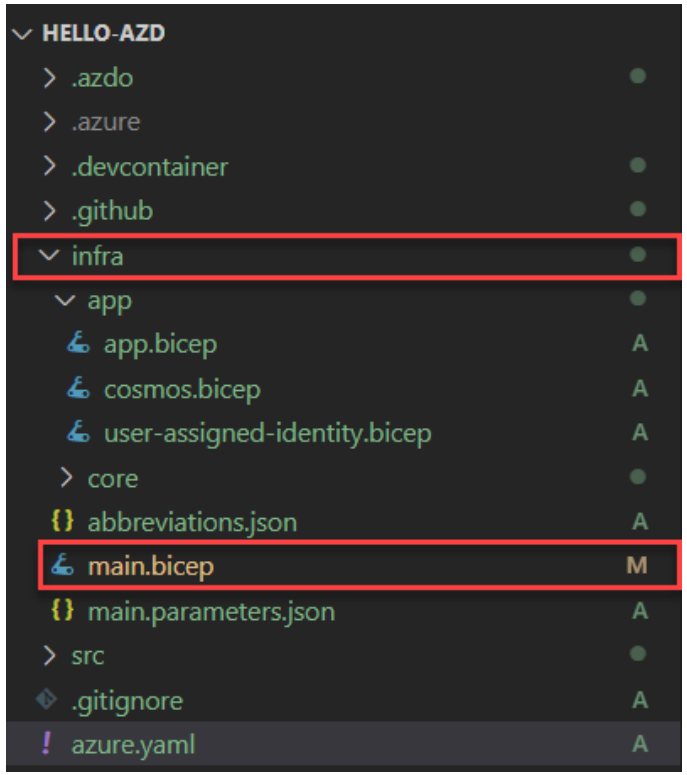
# Provisioning stage

The provisioning stage sets up the necessary Azure resources for your app, such as an App Service for hosting and a Storage Account for files, using infrastructure-as-code files from the template.

azd supports both Bicep and Terraform for infrastructure-as-code tasks.

By default, infrastructure-as-code files are stored in the infra folder, but this location can be customized.

azd searches for a main.bicep or main.tf file to act as the main orchestration file for the IaC process.



# Deployment stage

The app artifacts created during the packaging stage are deployed to the provisioned Azure resources.

azd uses configuration files in the template, such as `azure.yaml`, to determine how to deploy the app.

Environment variables and connection strings are configured to ensure the app can interact with the provisioned resources.

azure.yaml

```yaml
# yaml-language-server: $schema=https://raw.githubuser

name: azd-productcentral
metadata:
    template: azd-init@1.19.0
services:
    productcentral-backend:
        language: csharp
        project: ./ProductCentral.BackEnd
        host: function
    productcentral-frontend:
        language: csharp
        project: ./ProductCentral.FrontEnd
        host: appservice
```

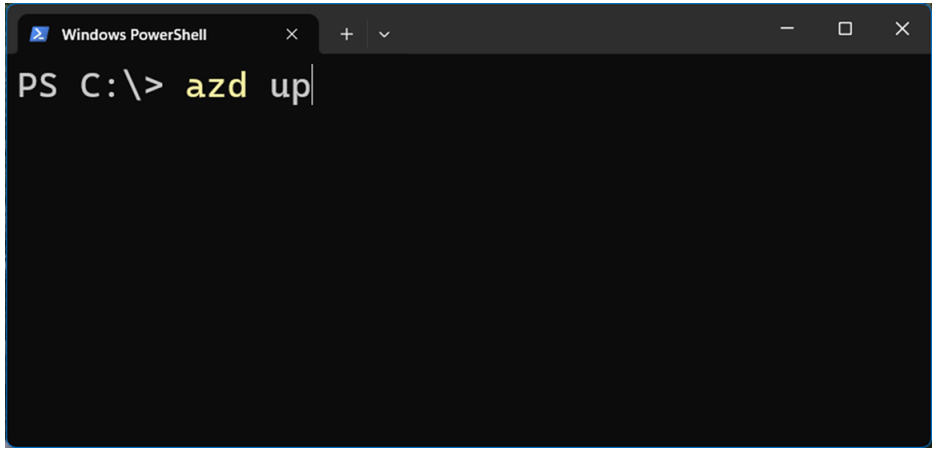
frontEnd.bicep

```bicep
resource frontEndAppServicePlan 'Microsoft.Web/serverfarms@2020-06-01
  name:'F1'
  }
  kind: 'windows'
}

resource frontEndAppService 'Microsoft.Web/sites@2021-02-01' = {
  name: appServiceName
  location: location
  kind: 'app'
  tags: {
    'azd-service-name': 'productcentral-frontend'
  }
  properties: {
    httpsOnly: true
    serverFarmId: frontEndAppServicePlan.id
    siteConfig: {
      netFrameworkVersion: 'v8.0'
    }
  }
  identity: {
    type: 'SystemAssigned'
  }
}

resource basicPublishingCredentials 'Microsoft.Web/sites/basicPublishin
  parent: frontEndAppService
  name: 'scm'
  properties: {
    allow: true
  }
}
```

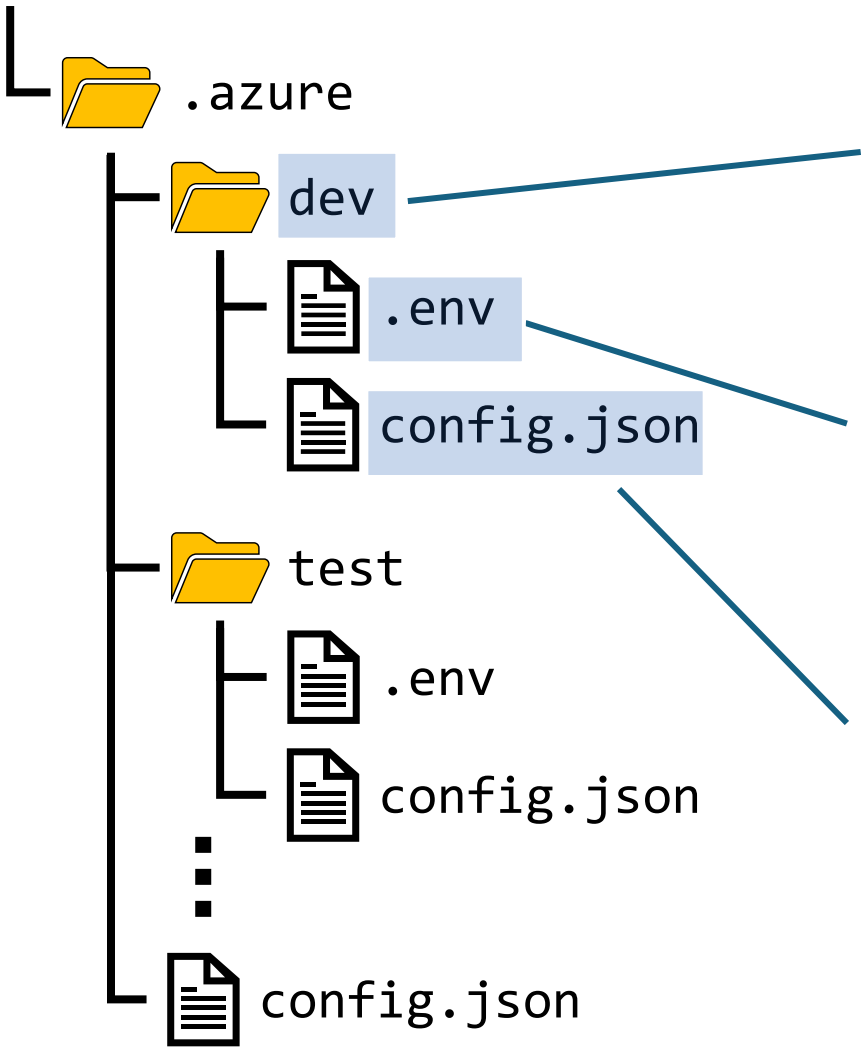
# Demo



# Environments

- An **environment** is a named, isolated configuration for a deployment of your app (e.g., dev, test, prod).
- Each environment contains its own values: **subscription**, **location**, **variables**, and **resource names**.
- Environments provide:
  - Isolation between development, testing, and production
  - Configuration management for separate settings
  - Collaboration (each developer can create a personal environment)
  - Reproducibility across stages

# Environments




Specific folder created by the command
`azd env new dev`

Contains environment-specific variables used by your application and during deployment

Used to drive settings that influence azd command behavior and features. This file isn't intended to be used directly by end users

# Environment variables

myAzure > .env

```
AZURE_ENV_NAME="myAzure"
AZURE_SUBSCRIPTION_ID="2c64efa2-a67e-434f-b039-8a2fe69589bf"
AZURE_LOCATION="eastus"
```

main.parameters.json > ...

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentParameters.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "environmentName": {
      "value": "${AZURE_ENV_NAME}"
    },
    "location": {
      "value": "${AZURE_LOCATION}"
    }
  }
}
```

- Environment variables are **key-value pairs** for environments like *dev*, *test*, or *prod*.
- They can be **local** or **remote**.
- Each azd environment has its own variables stored in **`.env`** files within the **`.azure`** folder.
- Azd can convert Bicep template outputs into environment variables.
- Avoid storing secrets in **`.env`** files; use **Azure KeyVault** instead.

# Hooks

- Hooks let you **run custom scripts** automatically at specific stages of the azd workflow.
- They allow developers to **extend**, **automate**, or **customize** provisioning, deployment, packaging, and more.
- **Triggered around** major azd commands (pre/post restore, pre/post provision…)
- Hooks are configured in `azure.yaml`, either at:
  - Root level → project-wide automation
  - Service level → per-service logic

```yaml
name: todo-nodejs-mongo
metadata:
  template: todo-nodejs-mongo@0.0.1-beta
hooks:
  prerestore: # Example of an inline script. (shell is required for inline scripts)
    shell: sh
    run: echo 'Hello'
  preprovision: # Example of external script (Relative path from project root)
    run: ./hooks/preprovision.sh
services:
  web:
    project: ./src/web
    dist: build
    language: js
    host: appservice
  api:
    project: ./src/api
    language: js
    host: appservice
```

# Advanced Features

## Extensions
Modular · Beta

- Official registry + custom sources
- Install: azd extension install <n>
- Auto-provision in Dev Containers
- v0.5.0: custom ServiceConfig settings

## .NET Aspire
Integration · GA Mar 2026

- Auto-detects AppHost at azd init
- Generates Bicep from Aspire manifest
- GA on Azure App Service (Mar 2026)
- Python apps supported via Aspire 1.3+

## AI Agent Integration
AI Foundry · MCP · Copilot

- AI Template Gallery: RAG, agents, MCP
- azd add → AI (OpenAI, AI Search)
- GitHub Copilot coding agent extension
- MCP tag in template gallery (May 2025)

## DevOps
GitHub Actions · Azure DevOps

- One command: azd pipeline config
- Auto-configures secrets & OIDC auth
- Custom pipeline definition files
- CI-ready: --no-prompt flag for automation

# Azure Developer CLI vs Azure CLI

## Azure Developer CLI

- High-level commands oriented around app lifecycle stages, such as provisioning and deployment
- A template system to define infrastructure as code and deployment configurations for your app
- Automated provisioning and deployment of app resources
- Built-in CI/CD pipeline setup for GitHub Actions or Azure Pipelines

**It streamlines development by managing resources and simplifying app building and deployment, allowing focus on application tasks.**

## Azure CLI

- Granular administrative control over Azure resources
- Support for scripting and task automation
- Integration with a wide range of Azure services and tools
- Resource management across many tenants, subscriptions, and environments

**Offers precise management of Azure resources for IT admins, DevOps, and developers, requiring specific service expertise.**

# Q&A?

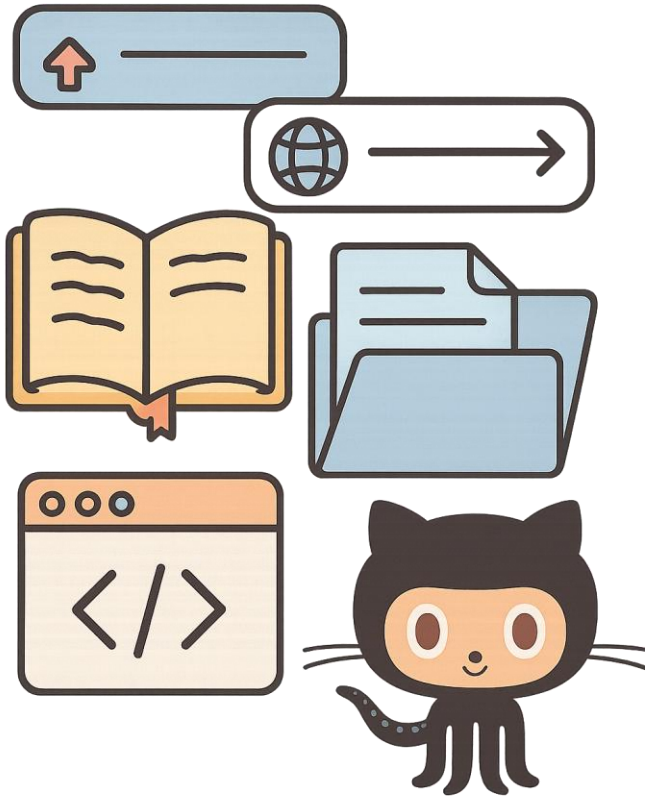
AZURE MEETUP PUGLIA

**Massimo Bonanni**

*Senior Technical Trainer*

massimo.bonanni@microsoft.com
aka.ms/maxlinkedin


Global Azure Puglia 2026 – 17 aprile

# References

- Azure Developer CLI (azd) | Microsoft Learn
- GitHub - Azure/azure-dev
- Welcome to Awesome Azure Developer CLI
- massimobonanni/AZD-ToDoList